Міністерство освіти і науки України

Національний технічний університет України

«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки

Кафедра обчислювальної техніки

Лабораторна робота №6

з дисципліни «Бази даних»

Виконав:
Степаненко Денис
студент групи ІМ-051
номер у списку групи: 16

Перевірив:

Хмельницький Арсеній Андрійович

Київ 2026

Короткий виклад вимог

Завдання:

Використати Prisma ORM для роботи з базою даних VideoHub. Описати схему в `schema.prisma`, створити міграції для зміни схеми, перевірити результат через Prisma Studio.

Що таке Prisma ORM:

Prisma — Object-Relational Mapping (ORM) інструмент для Node.js/TypeScript. Замість написання SQL-запитів вручну, Prisma генерує типобезпечний клієнт з `schema.prisma`. Міграції автоматично створюють SQL для зміни структури БД.

Що таке міграція:

Міграція — контрольована зміна структури БД. Prisma порівнює `schema.prisma` з поточним станом БД і генерує SQL-файл з потрібними змінами (CREATE TABLE, ALTER TABLE, DROP). Кожна міграція зберігається в `prisma/migrations/` — це історія змін схеми.

Поняття:

- **schema.prisma** — декларативний опис моделей БД (аналог SQL CREATE TABLE, але зручніший)
- **prisma migrate dev** — команда для створення та застосування міграції
- **prisma generate** — генерація TypeScript-клієнта з schema.prisma
- **prisma studio** — GUI для перегляду та редагування даних
- **@relation** — опис зв'язку між моделями (FK)
- **@map** — мапінг імені поля в БД (наприклад, `authorId` → `author_id`)

Ініціалізація Prisma

Створюємо Node.js-проєкт з Prisma:

```
{
  "name": "videohub-prisma",
  "version": "1.0.0",
  "devDependencies": {
    "prisma": "^6.0.0"
  },
```

```
  "dependencies": {
    "@prisma/client": "^6.0.0"
  }
}
```

Підключення до PostgreSQL через `.env` (пароль — плейсхолдер; реальний `.env` не комітиться):

```
DATABASE_URL="postgresql://postgres:password@localhost:5432/videohub?schema=public"
```

---

schema.prisma — схема VideoHub

Описуємо моделі VideoHub — **ту саму фінальну схему, що в Lab 2**: композитні первинні ключі Like/Subscription, TIMESTAMPTZ для часових міток, генерація UUID через `uuid_generate_v4()`:

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

model User {
  id          String   @id @default(dbgenerated("uuid_generate_v4()")) @db.Uuid
  email       String   @unique @db.VarChar(255)
  username    String   @unique @db.VarChar(100)
  password    String?  @db.VarChar(255)
  googleId    String?  @unique @map("google_id") @db.VarChar(255)
  description String?
  avatar      String?  @db.VarChar(500)
  banner      String?  @db.VarChar(500)
  createdAt   DateTime @default(dbgenerated("now()")) @map("created_at") @db.Timestamptz

  videos        Video[]
  comments      Comment[]
  likes         Like[]
  subscribers   Subscription[] @relation("SubscriberRelation")
  subscriptions Subscription[] @relation("ChannelRelation")

  @@map("User")
}

model Video {
  id          String   @id @default(dbgenerated("uuid_generate_v4()")) @db.Uuid
  title       String   @db.VarChar(255)
  description String?
  url         String   @db.VarChar(500)
  thumbnail   String?  @db.VarChar(500)
  views       Int      @default(0)
```

```
  createdAt   DateTime @default(dbgenerated("now()")) @map("created_at") @db.Timestamptz
  authorId    String   @map("author_id") @db.Uuid

  author      User     @relation(fields: [authorId], references: [id], onDelete: Cascade)
  comments    Comment[]
  likes       Like[]

  @@map("Video")
}

model Comment {
  id        String   @id @default(dbgenerated("uuid_generate_v4()")) @db.Uuid
  text      String
  createdAt DateTime @default(dbgenerated("now()")) @map("created_at") @db.Timestamptz
  userId    String   @map("user_id") @db.Uuid
  videoId   String   @map("video_id") @db.Uuid

  user      User     @relation(fields: [userId], references: [id], onDelete: Cascade)
  video     Video    @relation(fields: [videoId], references: [id], onDelete: Cascade)

  @@map("Comment")
}

model Like {
  userId    String   @map("user_id") @db.Uuid
  videoId   String   @map("video_id") @db.Uuid
  createdAt DateTime @default(dbgenerated("now()")) @map("created_at") @db.Timestamptz

  user      User     @relation(fields: [userId], references: [id], onDelete: Cascade)
  video     Video    @relation(fields: [videoId], references: [id], onDelete: Cascade)

  @@id([userId, videoId])

  @@map("Like")
}

model Subscription {
  subscriberId String   @map("subscriber_id") @db.Uuid
  channelId    String   @map("channel_id") @db.Uuid
  createdAt    DateTime @default(dbgenerated("now()")) @map("created_at") @db.Timestamptz

  subscriber User @relation("SubscriberRelation", fields: [subscriberId], references: [id],
onDelete: Cascade)
  channel    User @relation("ChannelRelation", fields: [channelId], references: [id],
onDelete: Cascade)

  @@id([subscriberId, channelId])

  @@map("Subscription")
}
```

Особливості schema.prisma:

- `@id @default(dbgenerated("uuid_generate_v4()"))` — первинний ключ; генерація UUID на рівні БД тим самим механізмом, що в Lab 2 (розширення `uuid-ossp`)
- `@db.Uuid` — нативний тип колонки UUID (16 байт), не TEXT
- `@db.VarChar(n)` — точний тип колонки як в Lab 2 (VARCHAR(255) тощо); без нього Prisma створила б TEXT
- `@db.Timestamptz` — час з часовою зоною (UTC всередині), як у фінальній схемі Lab 2
- `@@id([userId, videoId])` — **композитний первинний ключ**: пара і є ідентичність лайка — окремий сурогатний `id` не потрібен (те саме рішення, що в Lab 2)
- `@unique` — унікальне обмеження
- `@map("column_name")` / `@@map("table_name")` — мапінг імен (camelCase в коді → snake_case в БД)
- `@relation(fields: [fk], references: [pk], onDelete: Cascade)` — зв'язок FK з каскадним видаленням
- `String?` — nullable поле (знак питання)

Міграція 1: init

Початкова схема — створення всіх таблиць VideoHub:

```
npx prisma migrate dev --name init
```

SQL-файл `migration.sql` (згенерований зі схеми; CHECK-обмеження додані вручну — Prisma їх не виражає):

```sql
-- CreateTable
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";

CREATE TABLE "User" (
    "id" UUID NOT NULL DEFAULT uuid_generate_v4(),
    "email" VARCHAR(255) NOT NULL,
    "username" VARCHAR(100) NOT NULL,
    "password" VARCHAR(255),
    "google_id" VARCHAR(255),
    "description" TEXT,
    "avatar" VARCHAR(500),
    "banner" VARCHAR(500),
    "created_at" TIMESTAMPTZ NOT NULL DEFAULT now(),

    CONSTRAINT "check_auth_method" CHECK (password IS NOT NULL OR google_id IS NOT NULL),
    CONSTRAINT "User_pkey" PRIMARY KEY ("id")
);

CREATE TABLE "Video" (
    "id" UUID NOT NULL DEFAULT uuid_generate_v4(),
    "title" VARCHAR(255) NOT NULL,
    "description" TEXT,
    "url" VARCHAR(500) NOT NULL,
    "thumbnail" VARCHAR(500),
    "views" INTEGER NOT NULL DEFAULT 0,
    "created_at" TIMESTAMPTZ NOT NULL DEFAULT now(),
    "author_id" UUID NOT NULL,
```

```
    CONSTRAINT "Video_pkey" PRIMARY KEY ("id")
);

CREATE TABLE "Like" (
    "user_id" UUID NOT NULL,
    "video_id" UUID NOT NULL,
    "created_at" TIMESTAMPTZ NOT NULL DEFAULT now(),

    CONSTRAINT "Like_pkey" PRIMARY KEY ("user_id", "video_id")
);

CREATE TABLE "Subscription" (
    "subscriber_id" UUID NOT NULL,
    "channel_id" UUID NOT NULL,
    "created_at" TIMESTAMPTZ NOT NULL DEFAULT now(),

    CONSTRAINT "check_no_self_subscribe" CHECK (subscriber_id <> channel_id),
    CONSTRAINT "Subscription_pkey" PRIMARY KEY ("subscriber_id", "channel_id")
);

-- Унікальні індекси
CREATE UNIQUE INDEX "User_email_key" ON "User"("email");
CREATE UNIQUE INDEX "User_username_key" ON "User"("username");
CREATE UNIQUE INDEX "User_google_id_key" ON "User"("google_id");

-- Зовнішні ключі (ON DELETE CASCADE — як у Lab 2)
ALTER TABLE "Video" ADD CONSTRAINT "Video_author_id_fkey"
    FOREIGN KEY ("author_id") REFERENCES "User"("id") ON DELETE CASCADE ON UPDATE CASCADE;

ALTER TABLE "Like" ADD CONSTRAINT "Like_user_id_fkey"
    FOREIGN KEY ("user_id") REFERENCES "User"("id") ON DELETE CASCADE ON UPDATE CASCADE;

ALTER TABLE "Like" ADD CONSTRAINT "Like_video_id_fkey"
    FOREIGN KEY ("video_id") REFERENCES "Video"("id") ON DELETE CASCADE ON UPDATE CASCADE;

ALTER TABLE "Subscription" ADD CONSTRAINT "Subscription_subscriber_id_fkey"
    FOREIGN KEY ("subscriber_id") REFERENCES "User"("id") ON DELETE CASCADE ON UPDATE
CASCADE;

ALTER TABLE "Subscription" ADD CONSTRAINT "Subscription_channel_id_fkey"
    FOREIGN KEY ("channel_id") REFERENCES "User"("id") ON DELETE CASCADE ON UPDATE CASCADE;
```

Міграція відтворює фінальну схему Lab 2 один в один: композитні PK, TIMESTAMPTZ, CHECK-и, каскадні FK. Таблиці Comment — за тим самим зразком (id, text, created_at, два FK з CASCADE).

Міграція 2: add-video-duration

Додаємо поле `duration` (тривалість відео в секундах) до моделі Video:

```
model Video {
  // ... існуючі поля
```

```
  duration    Int?     @map("duration")    // нове поле
}
```

```
npx prisma migrate dev --name add_video_duration
```

Згенерований SQL:

```
ALTER TABLE "Video" ADD COLUMN "duration" INTEGER;
```

`Int?` — nullable (не всі відео мають відому тривалість при завантаженні). `ALTER TABLE ADD COLUMN` додає колонку до існуючої таблиці без втрати даних.

Міграція 3: add-video-category

Додаємо таблицю `Category` та зв'язок Video → Category (M:1):

```
model Category {
  id    Int    @id @default(autoincrement())
  name  String @unique @db.VarChar(100)

  videos Video[]

  @@map("Category")
}

model Video {
  // ... існуючі поля
  category    Category? @relation(fields: [categoryId], references: [id])
  categoryId  Int?      @map("category_id")    // нове поле FK
}
```

```
npx prisma migrate dev --name add_video_category
```

Згенерований SQL:

```
CREATE TABLE "Category" (
    "id" SERIAL NOT NULL,
    "name" VARCHAR(100) NOT NULL,
    CONSTRAINT "Category_pkey" PRIMARY KEY ("id")
);

CREATE UNIQUE INDEX "Category_name_key" ON "Category"("name");

ALTER TABLE "Video" ADD COLUMN "category_id" INTEGER;

ALTER TABLE "Video" ADD CONSTRAINT "Video_category_id_fkey"
```

```
    FOREIGN KEY ("category_id") REFERENCES "Category"("id")
    ON DELETE SET NULL ON UPDATE CASCADE;
```

`Category?` — nullable FK (відео може не мати категорії). `ON DELETE SET NULL` — при видаленні категорії, `category_id` відео стає NULL (не видаляється відео). `@default(autoincrement())` — автоінкремент для id категорії (SERIAL в PostgreSQL).

Prisma Studio

Перевірка даних через Prisma Studio — GUI для перегляду та редагування:

```
npx prisma studio
```

Prisma Studio відкривається в браузері на порту 5555. Показує всі таблиці з `schema.prisma`, дозволяє переглядати, додавати, редагувати та видаляти записи.

Скріншоти

schema.prisma — опис моделей VideoHub (частина 1: generator, datasource, User)

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

// ============================================
// МОДЕЛІ (відповідають фінальній схемі з Lab 2:
// композитні PK, TIMESTAMPTZ, uuid_generate_v4)
// ============================================

model User {
  id          String   @id @default(dbgenerated("uuid_generate_v4()")) @db.Uuid
  email       String   @unique @db.VarChar(255)
  username    String   @unique @db.VarChar(100)
  password    String?  @db.VarChar(255)
  googleId    String?  @unique @map("google_id") @db.VarChar(255)
  description String?
  avatar      String?  @db.VarChar(500)
  banner      String?  @db.VarChar(500)
  createdAt   DateTime @default(dbgenerated("now()")) @map("created_at") @db.Timestamptz

  videos        Video[]
  comments      Comment[]
  likes         Like[]
  subscribers   Subscription[] @relation("SubscriberRelation")
  subscriptions Subscription[] @relation("ChannelRelation")
```

schema.prisma — моделі Video, Comment, Like (частина 2)

```
model Video {
  description String?
  url         String   @db.VarChar(500)
  thumbnail   String?  @db.VarChar(500)
  views       Int      @default(0)
  duration    Int?
  createdAt   DateTime @default(dbgenerated("now()")) @map("created_at") @db.Timestamptz
  authorId    String   @map("author_id") @db.Uuid
  categoryId  Int?     @map("category_id")

  author      User     @relation(fields: [authorId], references: [id], onDelete: Cascade)
  category    Category? @relation(fields: [categoryId], references: [id])
  comments    Comment[]
  likes       Like[]

  @@map("Video")
}

model Comment {
  id        String   @id @default(dbgenerated("uuid_generate_v4()")) @db.Uuid
  text      String
  createdAt DateTime @default(dbgenerated("now()")) @map("created_at") @db.Timestamptz
  userId    String   @map("user_id") @db.Uuid
  videoId   String   @map("video_id") @db.Uuid

  user      User     @relation(fields: [userId], references: [id], onDelete: Cascade)
  video     Video    @relation(fields: [videoId], references: [id], onDelete: Cascade)

  @@map("Comment")
}

model Like {
  userId    String   @map("user_id") @db.Uuid
  videoId   String   @map("video_id") @db.Uuid
  createdAt DateTime @default(dbgenerated("now()")) @map("created_at") @db.Timestamptz

  user      User     @relation(fields: [userId], references: [id], onDelete: Cascade)
  video     Video    @relation(fields: [videoId], references: [id], onDelete: Cascade)

  @@id([userId, videoId])

  @@map("Like")
}
```

schema.prisma — моделі Subscription, Category (частина 3)

```
model Subscription {
  subscriberId String   @map("subscriber_id") @db.Uuid
  channelId    String   @map("channel_id") @db.Uuid
  createdAt    DateTime @default(dbgenerated("now()")) @map("created_at") @db.Timestamptz

  subscriber User @relation("SubscriberRelation", fields: [subscriberId], references: [id], onDelete: C
  channel    User @relation("ChannelRelation", fields: [channelId], references: [id], onDelete: Cascade

  @@id([subscriberId, channelId])

  @@map("Subscription")
}

// ==========================================
// МОДЕЛІ ДОДАНІ ЧЕРЕЗ МІГРАЦІЇ
// ==========================================

model Category {
  id    Int    @id @default(autoincrement())
  name  String @unique @db.VarChar(100)

  videos Video[]

  @@map("Category")
}
```

Міграція 1 — init: CREATE TABLE (частина 1)

```sql
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";

-- CreateTable
CREATE TABLE "User" (
    "id" UUID NOT NULL DEFAULT uuid_generate_v4(),
    "email" VARCHAR(255) NOT NULL,
    "username" VARCHAR(100) NOT NULL,
    "password" VARCHAR(255),
    "google_id" VARCHAR(255),
    "description" TEXT,
    "avatar" VARCHAR(500),
    "banner" VARCHAR(500),
    "created_at" TIMESTAMPTZ NOT NULL DEFAULT now(),

    CONSTRAINT "check_auth_method" CHECK (password IS NOT NULL OR google_id IS NOT NULL),
    CONSTRAINT "User_pkey" PRIMARY KEY ("id")
);

-- CreateTable
CREATE TABLE "Video" (
    "id" UUID NOT NULL DEFAULT uuid_generate_v4(),
    "title" VARCHAR(255) NOT NULL,
    "description" TEXT,
    "url" VARCHAR(500) NOT NULL,
    "thumbnail" VARCHAR(500),
    "views" INTEGER NOT NULL DEFAULT 0 CHECK (views >= 0),
    "created_at" TIMESTAMPTZ NOT NULL DEFAULT now(),
    "author_id" UUID NOT NULL,

    CONSTRAINT "Video_pkey" PRIMARY KEY ("id")
);
```

Міграція 1 — init: індекси та зовнішні ключі (частина 2)

```sql
CREATE TABLE "Like" (
    "user_id" UUID NOT NULL,
    "video_id" UUID NOT NULL,
    "created_at" TIMESTAMPTZ NOT NULL DEFAULT now(),

    CONSTRAINT "Like_pkey" PRIMARY KEY ("user_id", "video_id")
);

-- CreateTable
CREATE TABLE "Subscription" (
    "subscriber_id" UUID NOT NULL,
    "channel_id" UUID NOT NULL,
    "created_at" TIMESTAMPTZ NOT NULL DEFAULT now(),

    CONSTRAINT "check_no_self_subscribe" CHECK (subscriber_id <> channel_id),
    CONSTRAINT "Subscription_pkey" PRIMARY KEY ("subscriber_id", "channel_id")
);

-- CreateIndex
CREATE UNIQUE INDEX "User_email_key" ON "User"("email");

-- CreateIndex
CREATE UNIQUE INDEX "User_username_key" ON "User"("username");

-- CreateIndex
CREATE UNIQUE INDEX "User_google_id_key" ON "User"("google_id");

-- AddForeignKey
ALTER TABLE "Video" ADD CONSTRAINT "Video_author_id_fkey" FOREIGN KEY ("author_id") REFERENCES "User"("

-- AddForeignKey
ALTER TABLE "Comment" ADD CONSTRAINT "Comment_user_id_fkey" FOREIGN KEY ("user_id") REFERENCES "User"("

-- AddForeignKey
ALTER TABLE "Comment" ADD CONSTRAINT "Comment_video_id_fkey" FOREIGN KEY ("video_id") REFERENCES "Video

-- AddForeignKey
ALTER TABLE "Like" ADD CONSTRAINT "Like_user_id_fkey" FOREIGN KEY ("user_id") REFERENCES "User"("id") O

-- AddForeignKey
ALTER TABLE "Like" ADD CONSTRAINT "Like_video_id_fkey" FOREIGN KEY ("video_id") REFERENCES "Video"("id"
```

Міграція 2 — add-video-duration (ALTER TABLE)

```sql
-- AlterTable: додати поле duration до Video
ALTER TABLE "Video" ADD COLUMN "duration" INTEGER;
```

Міграція 3 — add-video-category (нова таблиця + FK)

MyWork > Lab_6 > prisma > migrations > 20261020190000_add_video_category > migration.sql

```sql
-- CreateTable: Category
CREATE TABLE "Category" (
    "id" SERIAL NOT NULL,
    "name" VARCHAR(100) NOT NULL,

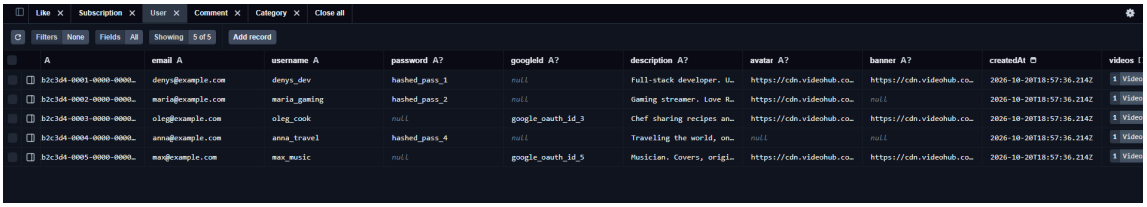
    CONSTRAINT "Category_pkey" PRIMARY KEY ("id")
);

-- CreateIndex
CREATE UNIQUE INDEX "Category_name_key" ON "Category"("name");

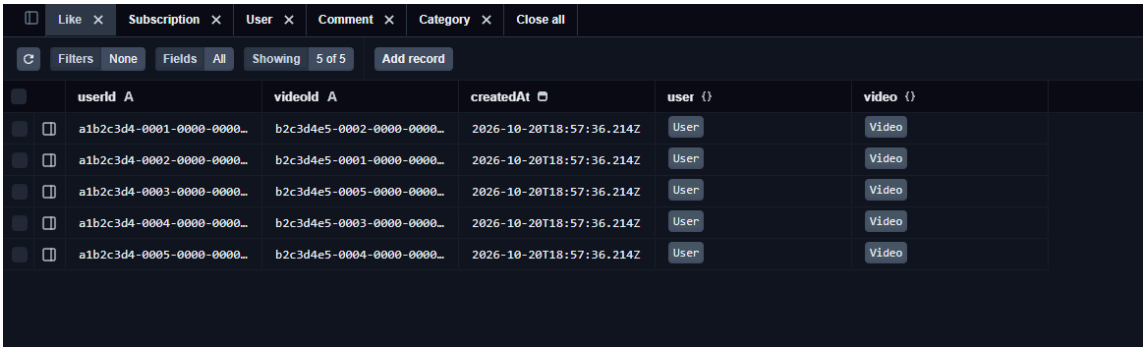
-- AlterTable: додати categoryId до Video
ALTER TABLE "Video" ADD COLUMN "category_id" INTEGER;

-- AddForeignKey
ALTER TABLE "Video" ADD CONSTRAINT "Video_category_id_fkey" FOREIGN KEY ("category_id") REFERENCES "Cat
```

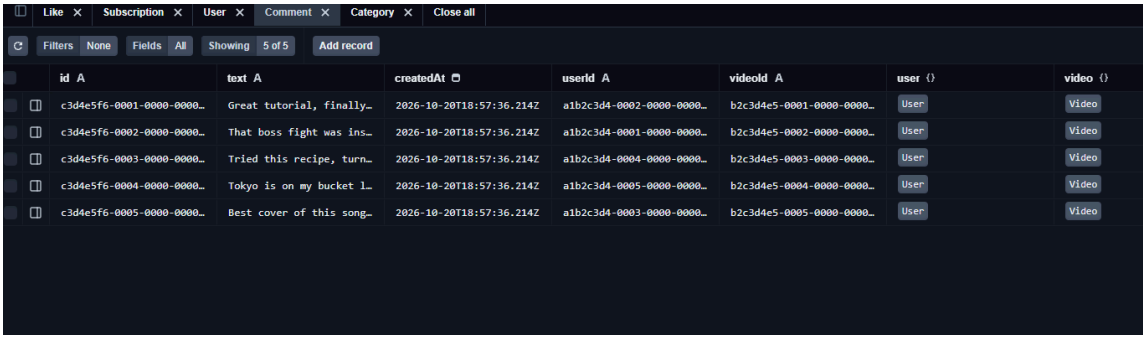
Prisma Studio — таблиця User з даними



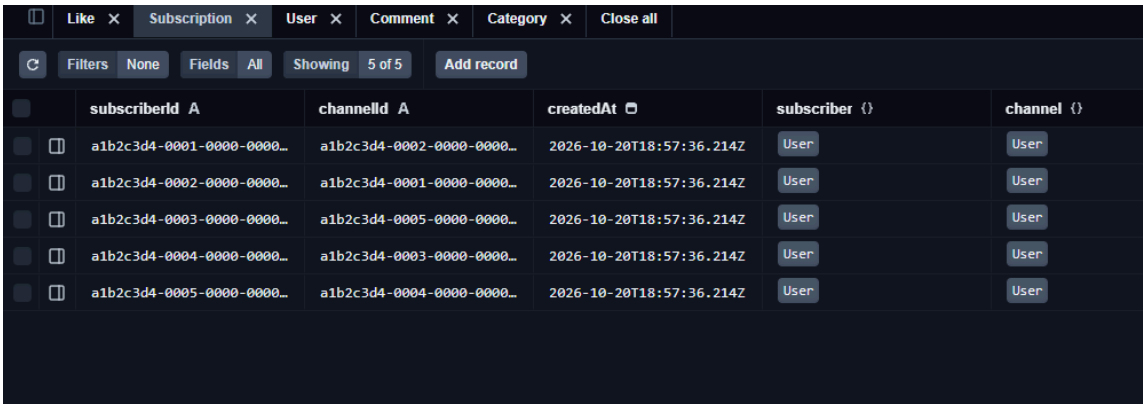
Prisma Studio — таблиця Like з даними (композитний PK: без колонки id)



Prisma Studio — таблиця Comment з даними



Prisma Studio — таблиця Subscription (subscriberId + channelId)



Походження бази даних

База даних **VideoHub** була взята з реального проєкту — відеохостинг-платформи на стеку Next.js + Prisma + PostgreSQL. Проєкт знаходиться в розробці, і схема БД продовжує дороблюватись в процесі проходження лабораторних робіт.

Як БД використовувалась у лабораторних:

- **Lab 1** — концептуальна модель (ER-діаграма) на основі схеми VideoHub
- **Lab 2** — DDL: фізична схема в PostgreSQL (CREATE TABLE, INSERT) на основі Prisma-схеми проєкту
- **Lab 3** — OLTP: CRUD-операції (SELECT, INSERT, UPDATE, DELETE) на тестових даних
- **Lab 4** — OLAP: агрегація, GROUP BY, HAVING, JOIN на тестових даних
- **Lab 5** — нормалізація: перевірка схеми на 1NF, 2NF, 3NF, виправлення через ALTER TABLE
- **Lab 6** — Prisma ORM: опис фінальної схеми в `schema.prisma`, міграції, Prisma Studio

Тестові дані:

Для тестування БД використовувались тестові дані з файлу `prisma/seed.sql` — 5 користувачів, 5 відео, 5 коментарів, 5 лайків, 5 підписок. Ці дані дозволяють перевірити всі операції (CRUD, агрегацію, JOIN, нормалізацію) на реалістичних прикладах.

Таблиця	Кількість	Приклади
User	5	denys_dev, maria_gaming, oleg_cook, anna_travel, max_music
Video	5	PostgreSQL Tutorial, Elden Ring, Pasta Carbonara, Tokyo, Wonderwall

Comment	5	коментарі до відео різних авторів
Like	5	лайки між користувачами (композитний PK — пара)
Subscription	5	підписки між користувачами (композитний PK — пара)

Чому БД з проєкту, а не з нуля:

Використання реальної схеми проєкту дає кілька переваг:

- Схема відповідає реальним вимогам і вже нормалізована
- Тестові дані реалістичні (користувачі, відео, коментарі)
- Лабораторні роботи логічно пов'язані — кожна наступна будується на попередній
- Prisma-схема описує той самий фінальний стан, що Lab 2 — обидва підходи (SQL і ORM) дають ідентичну структуру

Будь які припущення чи обмеження

- `DATABASE_URL` містить облікові дані PostgreSQL. В реальному проєкті `.env` не комітиться в Git (в `.gitignore`). Для лабораторної створено `.env.example` з плейсхолдером замість пароля.
- CHECK-обмеження (`check_auth_method`, `views >= 0`, `check_no_self_subscribe`) не виражаються в schema.prisma — вони додані вручну в SQL міграції. Це стандартна практика: Prisma описує структуру, спеціальні перевірки — на рівні БД.
- `@map` та `@@map` використовуються для мапінгу camelCase (Prisma/TypeScript) → snake_case (PostgreSQL). Наприклад, `authorId` в коді → `author_id` в БД.
- `onDelete: Cascade` на @relation — при видаленні батьківського запису автоматично видаляються дочірні (як ON DELETE CASCADE в SQL з Lab 2).
- `ON DELETE SET NULL` на Video→Category — при видаленні категорії відео не видаляється, `category_id` стає NULL. Це безпечніше за CASCADE для категорій.
- `@default(autoincrement())` для Category.id — SERIAL в PostgreSQL, автоінкремент. На відміну від UUID, числовий id зручніший для категорій (невеликий справочник).
- `dbgenerated("uuid_generate_v4()")` — генерація UUID на рівні БД через розширення `uuid-ossp` (той самий механізм, що в Lab 2). Альтернатива Prisma — `@default(uuid())` генерує `gen_random_uuid()` (вбудований в PostgreSQL 13+); обидва дають UUID v4.
- Prisma migrate dev автоматично: 1) порівнює schema.prisma з БД, 2) генерує SQL-міграцію, 3) застосовує до БД, 4) генерує клієнт. В продакшені використовується `prisma migrate deploy` (без генерації).
- Міграції зберігаються в `prisma/migrations/` — це історія змін схеми. Кожна міграція має timestamp + name + migration.sql. Дозволяє відкотити зміни або відтворити схему на новій БД.